ADVERSARIAL TRAINING FOR ROBUST IMAGE CLASSIFICATION
TRUSTWORTHY MACHINE LEARNING

1 INTRODUCTION

In this assignment, the task is to train an image classifier which works not only on clean images, but also on adversarially perturbed images. Normal models can get good clean accuracy, but they can fail when very small noise is added to the input. So, the main goal here is robustness. For this, I used adversarial training. Instead of training only on normal images, I created attacked versions of the images during training and trained the model on those harder examples. I kept the input images as raw float values in the range $[0, 1]$ and did not use ImageNet normalization, because the server also evaluates the model in the same input format.

2 APPROACH

My final model is based on ResNet-18. Since the task has 9 classes, I replaced the last fully connected layer with a new layer that outputs 9 logits. I used pretrained ImageNet weights as initialization, but the BatchNorm layers were adapted during training to the $[0, 1]$ input distribution. The training was done in two stages. First, I trained the model for 5 clean warm-up epochs. This helped the pretrained model adjust to the small 32×32 images and the new input range. During training, I used simple augmentation like random horizontal flip and random crop after padding. After the warm-up, I used PGD adversarial training for 95 epochs. For every batch, I first generated adversarial examples using the current model, and then trained the model on those adversarial images. The attack used for training was PGD-3 with L_∞ budget:

$$\epsilon = \frac{8}{255}, \quad \alpha = \frac{2}{255}$$

I used PGD-3 instead of PGD-7 during training because PGD-7 was slower and less stable with the learning rate used. PGD-3 gave a better balance between training time, clean accuracy, and robustness. For optimization, I used SGD with momentum. The learning rate was 0.05, momentum was 0.9, and weight decay was 5×10^{-4}. I also used cross entropy loss with label smoothing 0.1, so that the model does not become too overconfident. The learning rate was reduced using a multi-step scheduler during training. The main training settings are:

Model	ResNet-18
Classes	9
Input	32×32, range $[0, 1]$
Training	5 clean epochs + 95 PGD epochs
Attack	PGD-3, $\epsilon = 8/255$, $\alpha = 2/255$
Batch size	128
Optimizer	SGD, lr 0.05, momentum 0.9
Loss	Cross entropy + label smoothing

For validation, I split the data into 90% training and 10% validation. I checked clean validation accuracy after every epoch. For robustness, I used PGD-7 attack on a smaller validation subset of 512 images every few epochs. The best model was selected using a balanced score:

$$score = 0.5 \times clean\_accuracy + 0.5 \times robust\_accuracy$$

This was done because I wanted the final model to perform well on both clean and adversarial images, not only one of them. Finally, I saved the best model as `model.pt`.

3 Public Leaderboard Result

For the final submission, I generated a file called `model.pt`. This file contains the trained weights of my final robust classifier. The model takes one RGB image of size 32×32 and predicts one class out of 9 classes. My final selected method is:

```
ResNet-18 robust classifier
with clean warm-up and PGD-3 adversarial training
```

My best public leaderboard result was:

```
Public leaderboard score:  0.565813
```

This result was obtained by submitting the saved `model.pt` file to the evaluation server. I used this model because it gives a balance between clean accuracy and adversarial robustness. Clean-only training can perform well on normal images, but it can fail against small adversarial perturbations. PGD adversarial training makes the model more stable against such attacks.

4 Conclusion

From this task, I understood that robustness is very important in trustworthy machine learning. A model should not only work on clean images, but it should also be stable when small adversarial noise is added. In my solution, clean warm-up helped the pretrained ResNet-18 adapt to the dataset, and PGD adversarial training helped improve robustness. Overall, the final model uses a simple but effective training pipeline: ResNet-18, $[0, 1]$ inputs, data augmentation, clean warm-up, PGD-3 adversarial training, and balanced model selection. This gave a better trade-off between clean performance and adversarial robustness for the assignment.

GitHub Repository: `https://github.com/Jagadeeswar-reddy-c/TML26_X_03`